CS3211 Assignment 2

Overview

This assignment explores Software Transactional Memory (STM) implementation using concurrency control mechanisms in Go.

Data Structures

The STM maintains:

- `Memory Guard Channels: []chan GuardReq`

Each memory guard goroutine maintains:

- `Value: uint32`
- `Version: uint64`
- `LockedBy: int64 (-1 means unlocked)`

Requests sent by transactions contain the following:

- `Op: OperationType - (READ, WRITE, LOCK, VALIDATE, UNLOCK)`
- `TxnId: int64`
- `Value: uint32`
- `Version: uint64`
- `RespChan: chan GuardResp`

Responses sent by memory guards contain the following:

- `Ok: bool`
- `Value: uint32`
- `Version: uint64`

Each request made from the transaction to a memory guard is wrapped in a `GuardReq` structure. Similarly, the responses from memory guards are also wrapped in a `GuardResp` structure.

Algorithm

The STM implementation uses Optimistic Concurrency Control (OCC) model to manage transactional memory. This allows multiple transactions to execute concurrently and only checks for potential conflicts during the `COMMIT` phase. Each transaction follows a structured pipeline consisting of optimistic read and write, resource acquisition, version validation, and final commitment.

In the first phase (optimistic read and write), all `WRITE` operations are buffered in a local `writeSet`, and the updates are applied in the fourth phase (final commitment). For `READ` operations, the transaction first checks its local writeSet to implement "Read-Your-Own-Writes". If the address is not found, it fetches the current value and version from the corresponding memory guard and stores them in a local `readSet`. This playback phase allows the transaction

to maintain a consistent view of memory and build the necessary state to validate serializability during the commit stage without blocking other concurrent transactions.

Upon reaching the next phase, the transaction sorts all accessed addresses to prevent circular-wait deadlocks and attempts to acquire locks from the sharded guards. After securing locks, it enters a validation stage where it verifies that the versions in its `readSet` still match the guards' current state. If validation is successful, the transaction pushes its buffered writes, updates the global versions, and releases all locks to finalize the update. If a conflict or version mismatch is detected at any point, the transaction releases its acquired locks and restarts the retry loop, ensuring strict atomicity and isolation.

Concurrency Design in Go

This STM implementation adopts the actor-model architecture to manage how transactions access the memory. Each memory location is managed by a memory guard goroutine, which runs `simulateGuard`. Each of these guard goroutines manages their local `value`, `version`, and `lockedBy` variable. This design ensures isolation of the state variables of each memory address and prevents race conditions as only the guard goroutine itself can access the local data it holds.

Communication between the client goroutine and each memory guard is done via channels. Specifically, there is a dedicated `respChan` channel local to the ongoing transaction within the client goroutine that accepts responses sent back by the memory guard goroutines. On the other hand, each memory guard goroutine also has their own `reqChan` channel that receives requests from different transactions.

Concurrency

We claim that this implementation supports **Transaction Level** concurrency. Transactions can read the same memory address concurrently as `READ` requests are non-blocking. **Transaction Level** concurrency is also observed during validation, write and release phases as goroutines are spawned in fan in / fan out fashion to contact multiple guards concurrently if there are no overlapping `WRITE` operations. Transactions with overlapping write phases that modify the same memory location(s) are made to abort and retry, ensuring exclusive access to the winner and preventing race conditions.

Go Patterns Implemented

Our implementation employs several established Go concurrency patterns to ensure correctness and prevent resource leaks:

Context-Based Cancellation: All long-lived goroutines (guards and connection handlers) use select statements with `ctx.Done()` to enable graceful shutdown when the system terminates.

WaitGroup Synchronization: Every spawned goroutine is tracked through `wg.Add(1)` before creation and defer `wg.Done()` upon completion, ensuring the `Shutdown()` method can wait for all goroutines to terminate cleanly.

Fan-Out/Fan-In Pattern: Applied extensively during transaction commit phases to maximize concurrency. During validation, write, and release phases, we spawn concurrent worker goroutines (fan-out) to distribute work evenly across multiple guards, then collect all results through buffered channels (fan-in) before proceeding to the next phase.

Buffered Response Channels: All response channels are created with buffer size 1 to prevent deadlocks and cyclic dependencies if either the sender or receiver terminates unexpectedly.

Higher-Order Channels: We embed response channels within request structures (`RespChan chan GuardResp`), enabling direct point-to-point communication between transactions and guards without requiring shared state or central coordination, maintaining the actor-model isolation guarantees.

Lexical Scoping: Guard state variables (`value`, `version`, `acquiredBy`) are confined within the `runGuard()` function scope, ensuring only that specific goroutine can access them. Similarly, transaction state (`readSet`, `writeSet`, `commands`) is confined to the `handleConnection()` goroutine, eliminating the need for explicit synchronization.

Testing Methodology

- Basic Correctness: Single client verifies READ returns values from preceding WRITEs.
- Concurrent Reads: Multiple clients read identical addresses, verify consistency.
- Write Conflicts: Two clients write overlapping addresses, verify serializability (one commits, other retries).
- Read-Your-Own-Writes: READ after WRITE to the same address returns written value.
- Stress Test: 10 clients × 100 transactions with random patterns.

Fairness and Starvation

Our implementation provides no explicit fairness guarantees as transactions retry immediately upon failure without backoff, and while sorted acquisition prevents deadlock, it does not prevent starvation.

We considered several fairness mechanisms without implementing them:

Priority scheduling (tracking retry counts to prioritize starving transactions) adds significant complexity that STM systems typically delegate to applications; wait queues (FIFO ordering per guard) would ensure fairness but contradicts the optimistic concurrency model and requires substantial state management; and timeout-based preemption would violate serializability guarantees.

We argue that Go's channel FIFO behavior to provide reasonable fairness under moderate contention, and applications requiring stronger guarantees can implement exponential backoff or adaptive retry strategies externally.